

RELATÓRIO ACADÊMICO TAREFA 10: Mecanismos de Sincronização em OpenMP

Aluno: Thiago Jordão Melo da Costa **Data:** 19 de Junho de 2026

1. RESUMO

A API OpenMP fornece diversas formas para lidar com condições de corrida (*race conditions*), que ocorrem quando múltiplas *threads* tentam atualizar um recurso compartilhado simultaneamente. Este relatório avalia cinco métodos de sincronização utilizando a estimativa de π por Monte Carlo. Empregamos a função `rand_r()`, que é *thread-safe* (reentrante), pois mantém o estado de sua semente localmente em vez de depender de variáveis estáticas globais. O experimento contrasta o uso de contadores compartilhados protegidos por `critical` e `atomic` dentro de laços, contadores privados com consolidação final, e o uso da cláusula `reduction`.

2. IMPLEMENTAÇÃO EM C

O código a seguir executa os 5 cenários propostos. A fim de extrair a medição de tempo correta em código paralelo (*wall clock time*), utilizamos a função `omp_get_wtime()`.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

#define TOSSES 50000000

int main() {
    int thread_count = 4;
    double start, finish;
    omp_set_num_threads(thread_count);

    printf("Simulacao de PI com %d tosses e %d threads.\n\n", TOSSES,
thread_count);

    //
=====
    // VERSÃO 1: Contador Compartilhado + CRITICAL no laço
    //
=====

    long long hits_v1 = 0;
    start = omp_get_wtime();
    #pragma omp parallel
    {
        unsigned int seed = omp_get_thread_num() + time(NULL);
        #pragma omp for
        for (long long i = 0; i < TOSSES; i++) {
            double x = ((double)rand_r(&seed) / RAND_MAX) * 2.0 - 1.0;
            double y = ((double)rand_r(&seed) / RAND_MAX) * 2.0 - 1.0;
```

```

        if (x*x + y*y <= 1.0) {
            #pragma omp critical
            hits_v1++;
        }
    }
}
finish = omp_get_wtime();
printf("V1 (Compartilhado + critical) | Pi: %.5f | Tempo: %f s\n", 4.0
* hits_v1 / TOSSES, finish - start);

//
=====
// VERSÃO 2: Contador Compartilhado + ATOMIC no laço
//
=====
long long hits_v2 = 0;
start = omp_get_wtime();
#pragma omp parallel
{
    unsigned int seed = omp_get_thread_num() + time(NULL);
    #pragma omp for
    for (long long i = 0; i < TOSSES; i++) {
        double x = ((double)rand_r(&seed) / RAND_MAX) * 2.0 - 1.0;
        double y = ((double)rand_r(&seed) / RAND_MAX) * 2.0 - 1.0;
        if (x*x + y*y <= 1.0) {
            #pragma omp atomic
            hits_v2++;
        }
    }
}
finish = omp_get_wtime();
printf("V2 (Compartilhado + atomic) | Pi: %.5f | Tempo: %f s\n", 4.0
* hits_v2 / TOSSES, finish - start);

//
=====
// VERSÃO 3: Contador Privado + CRITICAL (Apenas no final)
//
=====
long long hits_v3 = 0;
start = omp_get_wtime();
#pragma omp parallel
{
    long long local_hits = 0;
    unsigned int seed = omp_get_thread_num() + time(NULL);
    #pragma omp for
    for (long long i = 0; i < TOSSES; i++) {
        double x = ((double)rand_r(&seed) / RAND_MAX) * 2.0 - 1.0;
        double y = ((double)rand_r(&seed) / RAND_MAX) * 2.0 - 1.0;
        if (x*x + y*y <= 1.0) local_hits++;
    }
    #pragma omp critical
    hits_v3 += local_hits;
}

```

```
    finish = omp_get_wtime();
    printf("V3 (Privado + critical)          | Pi: %.5f | Tempo: %f s\n", 4.0
* hits_v3 / TOSSES, finish - start);

    //
=====
    // VERSÃO 4: Contador Privado + ATOMIC (Apenas no final)
    //
=====

    long long hits_v4 = 0;
    start = omp_get_wtime();
    #pragma omp parallel
    {
        long long local_hits = 0;
        unsigned int seed = omp_get_thread_num() + time(NULL);
        #pragma omp for
        for (long long i = 0; i < TOSSES; i++) {
            double x = ((double)rand_r(&seed) / RAND_MAX) * 2.0 - 1.0;
            double y = ((double)rand_r(&seed) / RAND_MAX) * 2.0 - 1.0;
            if (x*x + y*y <= 1.0) local_hits++;
        }
        #pragma omp atomic
        hits_v4 += local_hits;
    }
    finish = omp_get_wtime();
    printf("V4 (Privado + atomic)          | Pi: %.5f | Tempo: %f s\n", 4.0
* hits_v4 / TOSSES, finish - start);

    //
=====
    // VERSÃO 5: Cláusula REDUCTION (Mais produtiva e performática)
    //
=====

    long long hits_v5 = 0;
    start = omp_get_wtime();
    #pragma omp parallel
    {
        unsigned int seed = omp_get_thread_num() + time(NULL);
        #pragma omp for reduction(+:hits_v5)
        for (long long i = 0; i < TOSSES; i++) {
            double x = ((double)rand_r(&seed) / RAND_MAX) * 2.0 - 1.0;
            double y = ((double)rand_r(&seed) / RAND_MAX) * 2.0 - 1.0;
            if (x*x + y*y <= 1.0) hits_v5++;
        }
    }
    finish = omp_get_wtime();
    printf("V5 (Clausula reduction)        | Pi: %.5f | Tempo: %f s\n", 4.0
* hits_v5 / TOSSES, finish - start);

    return 0;
}
```

3. COMPARAÇÃO E DISCUSSÃO

Abaixo, apresentamos os resultados de tempo de execução e valor de π estimado para as 5 versões, obtidos a partir de uma execução com 4 *threads* e 50.000.000 de iterações (TOSSES):

Versão	Mecanismo de Sincronização	Abordagem de Acúmulo	Tempo de Execução (s)	Valor de π Estimado
V1	Contador Compartilhado + <code>critical</code> no laço	Atualização direta na variável global	6.046332	3.14172
V2	Contador Compartilhado + <code>atomic</code> no laço	Atualização direta na variável global	1.748739	3.14150
V3	Contador Privado + <code>critical</code> no final	Variável local acumulada e somada no final	0.151081	3.14154
V4	Contador Privado + <code>atomic</code> no final	Variável local acumulada e somada no final	0.146572	3.14154
V5	Cláusula <code>reduction</code>	Acúmulo privado com redução final automática	0.157255	3.14154

As Versões 1 e 2 (Variáveis Compartilhadas Atualizadas Constantemente) O uso de `#pragma omp critical` na V1 exige que o compilador crie um bloqueio onde apenas uma *thread* por vez pode atualizar a variável `hits_v1`. Com 50 milhões de iterações, essa serialização severa causará um desempenho abismal (levando mais de 6 segundos). A diretiva `#pragma omp atomic` na V2 instrui o sistema a usar operações especiais de hardware (*load-modify-store*) que são muito mais rápidas e afetam apenas aquela instrução específica, sem bloquear as *threads* de forma genérica. Apesar de a V2 ser visivelmente mais rápida que a V1 (~1.75 segundos), a alta frequência de atualizações à mesma linha de cache na memória ainda causará lentidão massiva devido à contenção no barramento de memória (o chamado *ping-pong* de linhas de cache).

As Versões 3 e 4 (Contadores Privados com Atualização Tardia) Nas versões V3 e V4, a acumulação de *hits* ocorre em variáveis locais (`local_hits`), armazenadas diretamente nos registradores ou pilhas privadas (*stacks*) das *threads*. Desse modo, o programa roda na máxima velocidade das unidades aritméticas durante todo o laço. O uso das construções `critical` e `atomic` ocorre **apenas uma vez por thread** ao final da sub-rotina para somar as contribuições no total global. Ambas oferecem velocidade excepcional (~0.15 segundos), superando as V1 e V2 em cerca de 40 vezes.

A Versão 5 (Produtividade e Eficiência via `reduction`) A V5 utiliza a cláusula `reduction(+:hits_v5)`. Nos bastidores, o compilador do OpenMP cria cópias privadas de `hits_v5` para cada *thread* com um escopo particular, inicializa-as em 0 (elemento neutro da adição), consolida-as ao fim do laço e protege o cálculo global como se existisse um bloco `critical` automático. **A V5 atinge o ápice da relação desempenho/produtividade:** entrega a mesma alta velocidade das V3 e V4 (~0.15 segundos), mas através de um código infinitamente mais conciso, protegendo o desenvolvedor de esquecer a criação ou inicialização de dados na memória paralela.

4. ROTEIRO DE APLICABILIDADE: QUANDO UTILIZAR QUAL MECANISMO?

Com base na arquitetura do OpenMP, propõe-se o seguinte fluxograma decisório para uso de exclusão mútua:

1. Cláusula **reduction**:

- **Aplicação:** A primeira opção.
- **Quando usar:** Ao aplicar repetidamente o mesmo operador matemático e comutativo (como **+**, *****, **MAX**, operações lógicas de bit) sobre uma sequência gerando um único resultado consolidado.
- **Por quê:** O compilador gerencia automaticamente a privatização e o somatório sem introduzir gargalos (*overheads*).

2. Diretiva **atomic**:

- **Aplicação:** A segunda opção.
- **Quando usar:** Quando a modificação a recursos globais trata-se apenas de incrementos rápidos do tipo **x <op>= expr** ou **x++**, não sendo um cenário passível da cláusula **reduction**.
- **Por quê:** Usa controle atômico no próprio processador, o que é drasticamente mais barato do que levantar as defesas pesadas do sistema operacional necessárias numa região crítica convencional.

3. Diretiva **critical** (sem nome):

- **Aplicação:** Para blocos complexos e pouco acessados.
- **Quando usar:** Se a atualização da variável englobar múltiplas linhas de código ou funções inteiras complexas.
- **Cuidado:** Todas as regiões **critical** sem nome no programa agem sob **o mesmo bloqueio global**. Se a *Thread A* entra em um **critical**, a *Thread B* não consegue acessar **nenhum** outro bloco **critical** sem nome no programa inteiro.

4. Diretivas **critical(nome)**:

- **Aplicação:** Regiões críticas distintas que precisam executar em paralelo.
- **Quando usar:** Para contornar a limitação de estrangulamento da diretiva acima. Se um bloco acessa a lista A e outro bloco acessa a lista B, eles podem ser nomeados **critical(A)** e **critical(B)**. As *threads* executarão ambas simultaneamente.
- **Cuidado:** Os nomes das seções são avaliados de modo estático durante a compilação, ou seja, são imutáveis em tempo de execução. Além disso, evite alinhar regiões críticas nomeadas uma dentro da outra (aninhamento) para evitar os severos riscos de *deadlock*.

5. Locks Explícitos (**omp_lock_t**):

- **Aplicação:** Foco em dados, não em código. *Fine-grained locking*.
- **Quando usar:** Se a granularidade ou a identificação do recurso a ser travado só é descoberta dinamicamente **em tempo de execução**, ou para controle complexo de acesso a estruturas de dados interligadas (como listas encadeadas, árvores, dequeus dinâmicos).
- **Por quê:** Pode-se alocar um vetor com **N** locks (usando **omp_init_lock** e **omp_set_lock**) e proteger exclusivamente o exato objeto visado (como a respectiva fila de mensagens para a thread **Q**), conferindo total paralelismo sem que outras áreas do programa sofram pausas desnecessárias. Exigem o maior esforço do programador para evitar vazamento de memória e travamentos lógicos.

